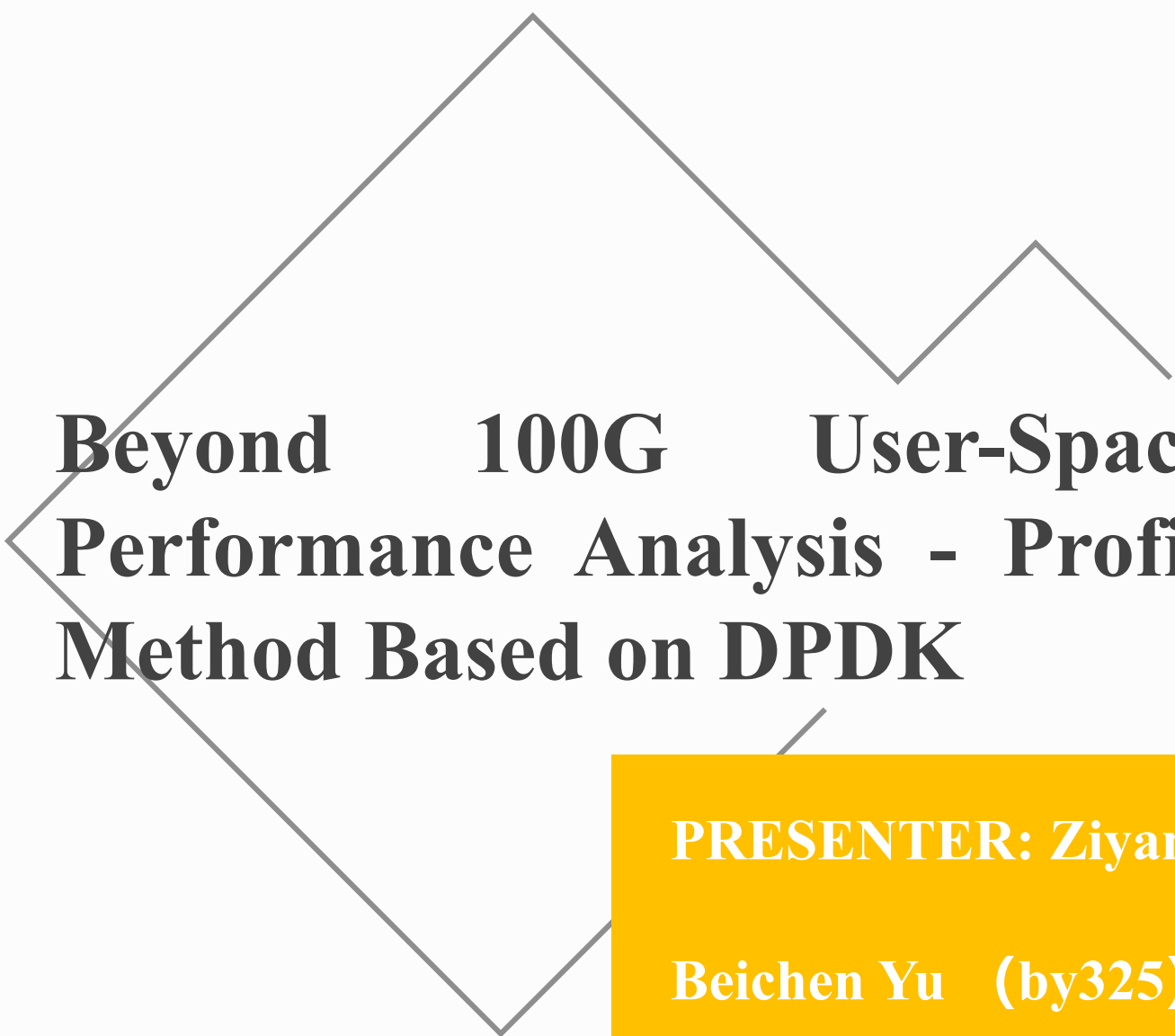
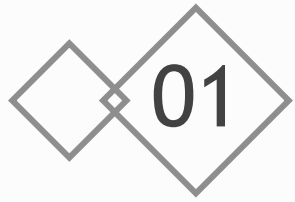




Beyond 100G User-Space Networking Stack Performance Analysis - Profiling-Driven Optimization Method Based on DPDK

PRESENTER: Ziyang Chen (zc579)

Beichen Yu (by325) 2026/4/29



Motivation

Motivation: Packet Processing in the Nanosecond Era

- Infrastructure (Hardware) is rapidly transitioning from 100G to 400G, even targeting 800G and more
- The demand within data centers for extremely low latency and exceptionally high throughput is pushing the limits of existing user-space network stacks
- Under a 100G full-load scenario for 64-byte minimum packets, packets arrive at the NIC every 6.72 nanoseconds, leaving a processing budget of approximately 14.11 CPU cycles @ 2.1 GHz

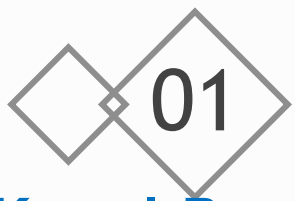
The Reality Gap: A performance gap of 6+ times

- Our Baseline: Single-core throughput: 24.01 Mpps
- 100G's Theoretical Maximum Value: 148.8 Mpps

Core Research Questions

- Where does this performance gap originate?
- What are the true bottlenecks in the user-space network stack?
- Can we identify and eliminate these bottlenecks through profiling-driven optimization method and provide some useful insights for optimizing user-space network stack?





Related Work

Kernel Bypass (e.g., DPDK): Eliminates the overhead about kernel interrupts and context switches through the use of a Polling Mode Driver (PMD), which **eliminates the macro-level bottleneck of kernel-to-user space data transfer**, serving as a fundamental component for building high-speed networks

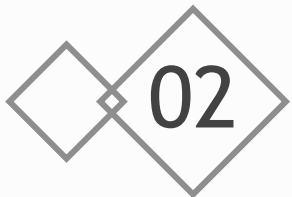
Advanced User-space Protocol Stack (e.g., Demikernel): Constructs a queue-oriented Library OS (LibOS) that shields the underlying hardware differences of DPDK and RDMA; Reduces context-switching overhead to approximately 12 CPU cycles by utilizing Rust coroutines - suitable for microsecond-scale data center networks

Efficient CPU scheduling mechanism (e.g. Caladan): Resolves resource interference issues between latency-critical (LC) tasks and best-effort (BE) tasks; Achieves core reallocation at the microsecond scale and reduces tail latency from 580ms (milliseconds) to 52μs (microseconds)

Research Gaps - Microsecond level Optimization and Black-box Datapath

Most optimizations in current state-of-the-art architectures **operate at the microsecond level**, whereas the single-packet processing budget at 100G line rate is merely 6.72 nanoseconds

Existing approaches focus on optimizing **macro-level** architecture and scheduling, without conducting an in-depth analysis of the specific processing paths of data packets



Methodology - Overview

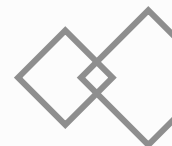
Project Goal: Shifting from macro-level scheduling to micro-level analysis. **By leveraging a profiling-driven approach to conduct an in-depth analysis of the Datapath**, we identify and pinpoint bottlenecks of the current user-space network and trying to give useful advice to high-speed user-space network stack design.

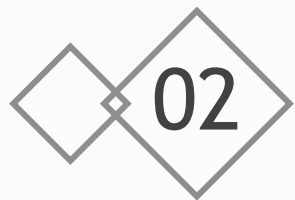
Experimental Environment

- 100G NIC: By employing a combination of 100G hardware and minimal 64-byte packets, the system's packet-processing rate per second is pushed to the limit (about 115Mpps) in order to expose concurrent processing bottlenecks as the system scales toward 400G
- 2.1GHz CPUs

Five-Stage Datapath Pipeline Decomposition based on DPDK User-space Echo Server

- RX: Transfers the packet from the NIC to memory (`rte_eth_rx_burst()`)
- Parse: Locates and extracts the Ethernet, IPv4, and UDP headers
- Rewrite: Swaps the source and destination IP/MAC addresses
- Checksum: Recalculates the checksum (potentially the biggest bottleneck)
- TX: Places the processed packet back into the transmit queue (`rte_eth_tx_burst()`)
- **Other (Architectural Friction): Represents system-level overhead** outside the explicit processing loop, which **includes empty polling, multi-core synchronization locks, cache/memory contention, and host-NIC interaction costs** that emerge under extreme loads/throughputs





Methodology - Overview

Core Metrics - Macro and Detailed

total_cycles/pkt

Total measured hot-loop cycles divided by received packets. This is the main CPU efficiency metric.

rx_cycles/pkt

Cycles spent in `rte_eth_rx_burst()`, normalized per packet.

parse_cycles/pkt

Cycles to locate Ethernet/IPv4/UDP headers from each mbuf.

rewrite_cycles/pkt

Cycles to swap MAC addresses, IP addresses, and UDP ports for echo.

checksum_cycles/pkt

checksum recomputation cost, offload preparation cost, or preserve-path overhead

tx_cycles/pkt

Cycles to submit packets to NIC: direct `tx_burst` or `tx_buffer` enqueue/flush.

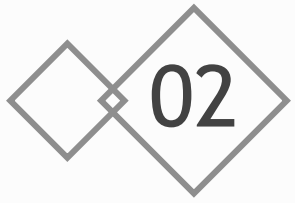
total_mpps

The number of packets processed by the system per second (in millions). The core performance benchmark

other_cycles/pkt

Residual: total minus RX/Parse/Rewrite/Checksum/TX. Includes empty polling, loop control, timer reads, stats updates, report checks, TX-buffer control, and small-batch overhead.





Methodology

User Stack Basic implementation based on DPDK

Kernel Bypass: Leverages the DPDK framework to take direct control of the NIC, thereby eliminating the overhead associated with interrupts and system calls in the traditional kernel protocol stack.

Zero-copy: Relies on `rte_mempool` for packet memory management. Throughout the entire data path from reception to transmission, only pointers are passed, eliminating any CPU-based memory copy operations.

Run-to-completion: A single CPU core independently handles the entire lifecycle of a packet, eliminating the friction associated with thread scheduling and context switching.

Optimization Strategy: Each optimization removes one bottleneck and exposes the next from 1 to 6

1. Multi-core implementation: RSS + multi-worker server, scaling from 1/2/4/6/8 cores
2. Checksum optimization: software vs offload vs preserve
3. Fast client: eliminate traffic generator starvation
4. Ring/Burst size: use larger RX/TX rings and larger burst size
5. TX-buffer: batch TX submission with `rte_eth_tx_buffer()`
6. RX prefetch: prefetch packet data after RX burst before parsing



Step 1: Multi-core Implementation

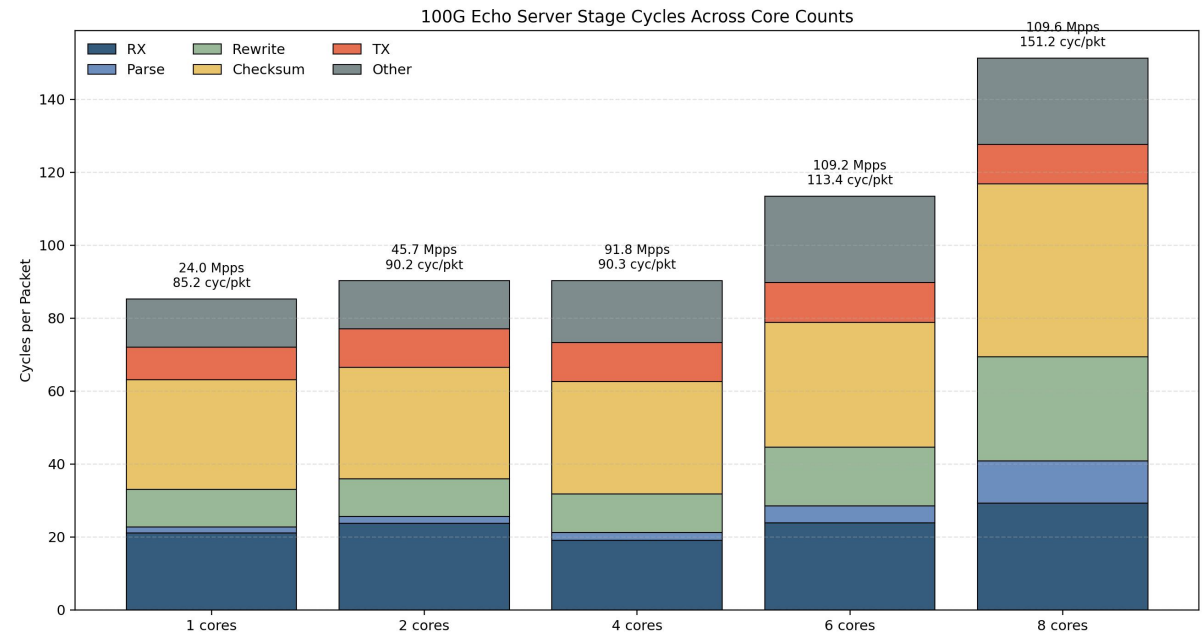
RSS and multiple workers establish the baseline scaling behavior

Implementation

- Multi-core echo server
- One RX/TX queue per worker
- RSS distribute IPv4/UDP flows
- Evaluate per-second stage profiling

Finding

1 → 6 cores scale reasonably, but 8 cores begins to degrade. This indicates that adding cores eventually increases shared-resource and polling overhead instead of useful throughput. Also under this setting, checksum is the biggest bottleneck.



Step 2: Checksum Optimization

Checksum recomputation was the first clear CPU bottleneck

Implemented modes

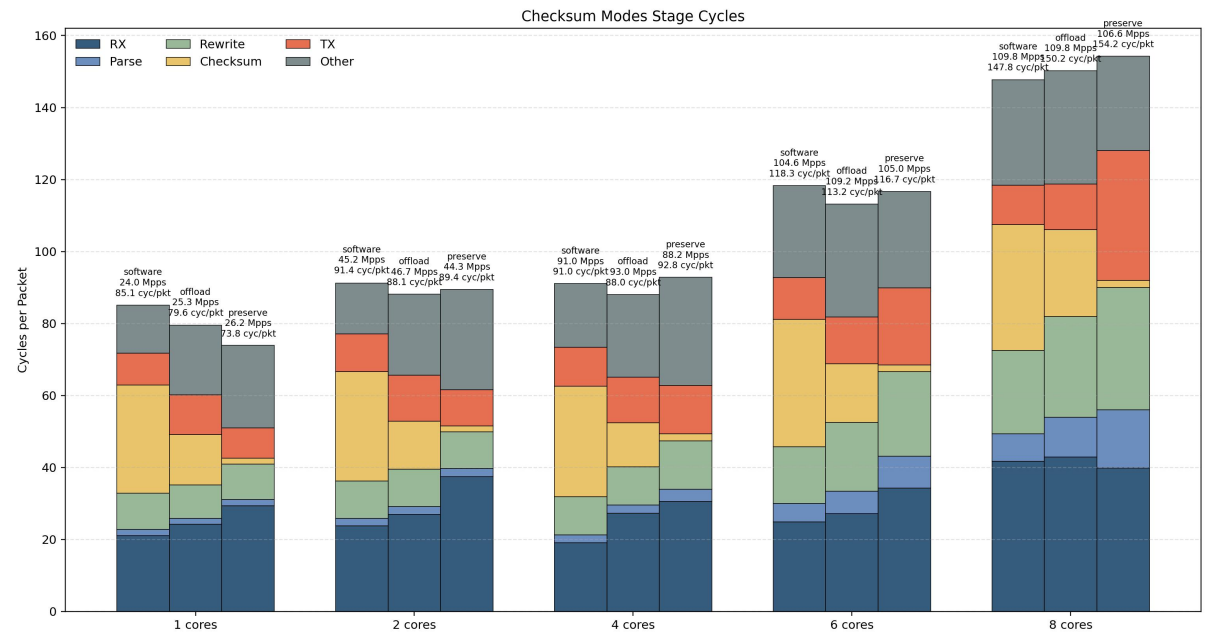
- software: recompute IPv4/UDP checksums
- offload: NIC TX checksum offload by DPDK
- preserve: keep checksum fields after symmetric swaps

Why preserve is valid here

The echo server only swaps source/destination MAC, IP, and UDP ports without changing length or payload. This ONLY works for this benchmark.

Result

After checksum optimization, the server datapath became much faster, so the bottleneck moved away from per-packet checksum computation and started to expose packet movement and batching issues. Therefore, we then tuned RX/TX ring size and burst size.



Step 3: Ring and Burst Size

Larger rings and bursts stabilize high-rate packet movement

Change

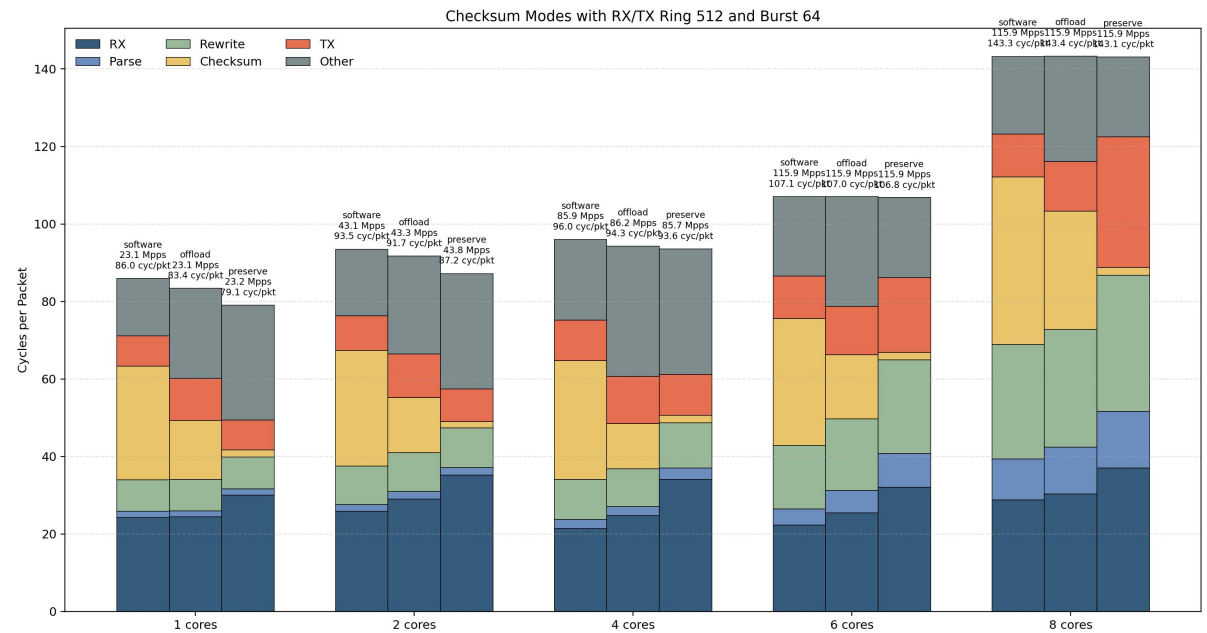
- RX ring size: 128 --> 512
- TX ring size: 128 --> 512
- BURST_SIZE: 32 --> 64

Why this matters

Small rings and bursts can create drops or unstable short windows under 100G load. Larger descriptors and larger burst size improve buffering and amortize per-call overhead.

Result

The new-burst experiment removes drops in stable windows and gives a cleaner baseline for later fast-client and TX/RX optimizations. But it doesn't show clear improvement on total cycles.



Step 4: Fast Client

A faster client made checksum benefits visible

Problem

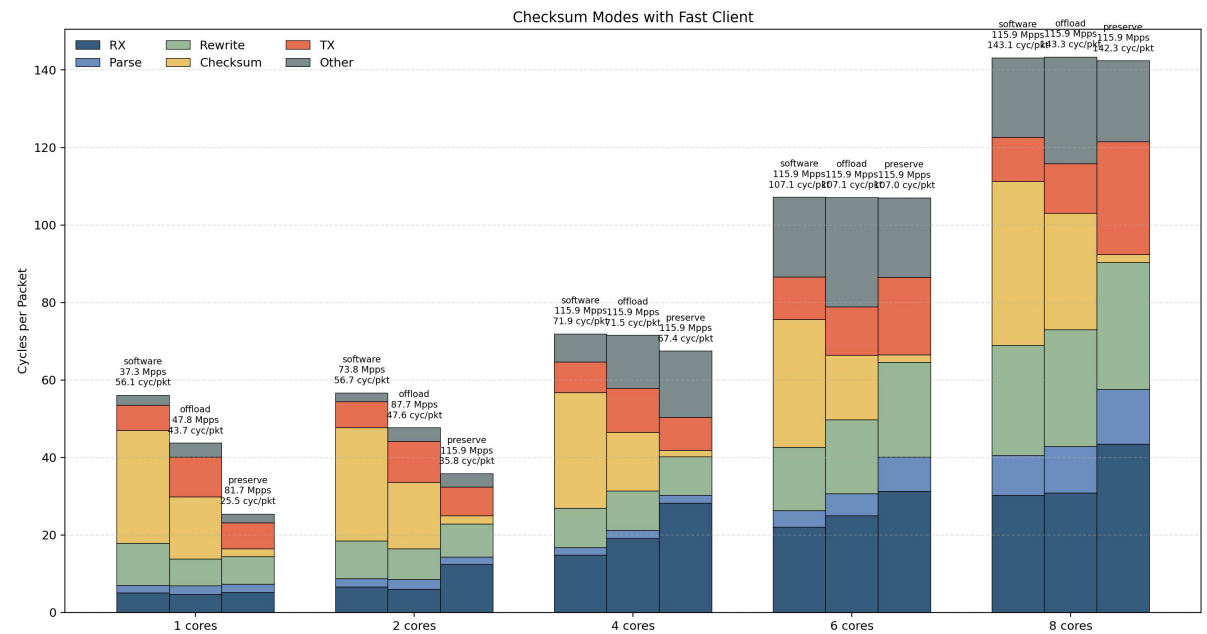
After checksum preserve, packet processing became faster. The original client could not always feed packets fast enough, causing empty polling and high Other.

Implementation

- Pre-generated packet templates
- Bulk mbuf allocation
- 8-worker fast client for server experiments
- Goal: make server-side bottlenecks visible

Result

Preserve mode reaches ~115.9 Mpps, which is saturation point in our current experiment, with 2 cores and shows much lower total cycles at low core counts. And the transmit stage becomes a new bottleneck under 6/8 cores.



Step 5: TX Buffer

After checksum and input bottlenecks, TX became visible

Implementation

A separate TX-buffer server keeps checksum logic unchanged but replaces direct `rte_eth_tx_burst()` with `rte_eth_tx_buffer()`, which helps create larger effective TX batch.

Key 8-core preserve comparison

Direct TX:

total = 142.3 cycles/pkt

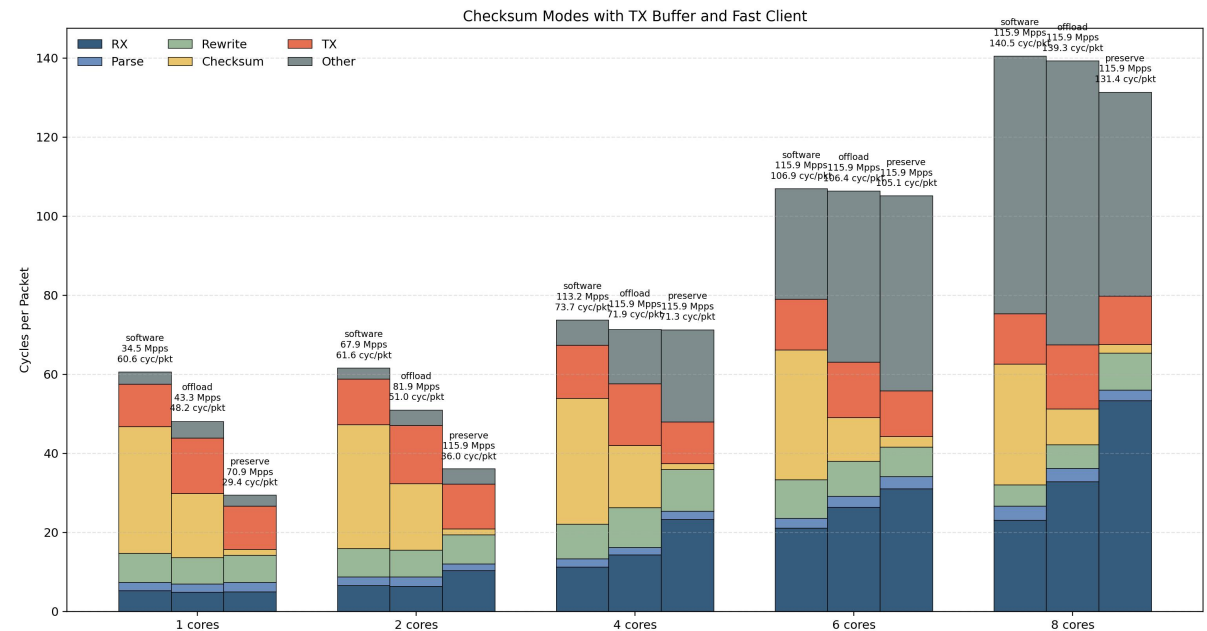
tx = 29.2 cycles/pkt

TX-buffer:

total = 131.4 cycles/pkt

tx = 12.3 cycles/pkt

Conclusion: TX-buffer is useful at high core counts, but it exposes more residual overhead in RX and Other.



Step 6: RX Prefetch

Prefetch reduces RX cost but does not solve high-core Other

Implementation

A separate TX-buffer + RX-prefetch server prefetches packet data for a future packet after rx_burst() and before parsing the current packet.

8-core preserve comparison

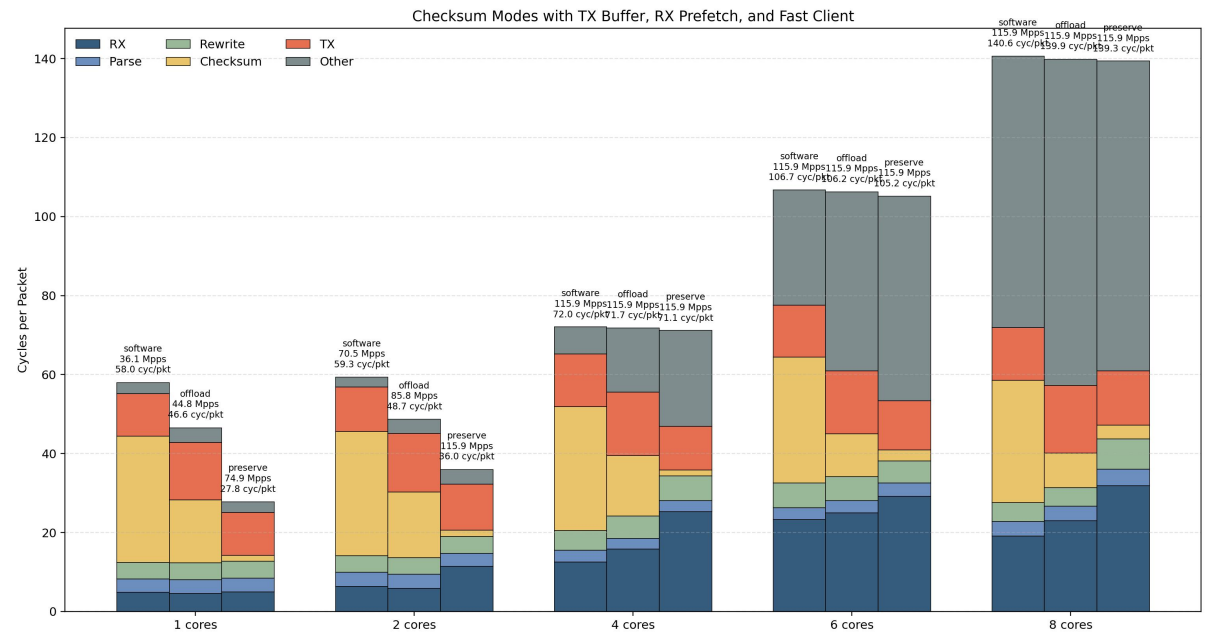
TX-buffer only:

rx = 53.3 cycles/pkt
other = 51.5 cycles/pkt

TX-buffer + prefetch:

rx = 31.9 cycles/pkt
other = 78.4 cycles/pkt

Conclusion: RX data access improved, but line-rate saturation and small-batch control overhead remain.



Why Other Is High in Multi-core Runs

4-core vs 6-core preserve mode after TX-buffer + RX prefetch

4-core preserve

rx_mpps = 115.9
other_cycles/pkt = 24.3
empty_poll_ratio = 0.001
empty_cycles/pkt = 0.0
avg_burst = 11.20
nonempty_cycles/pkt = 71.1

At 4 cores, throughput is already saturated with almost no empty polling. Batches are larger, so control overhead is amortized better.

6-core preserve

rx_mpps = 115.9
other_cycles/pkt = 51.8
empty_poll_ratio = 0.214
empty_cycles/pkt = 11.9
avg_burst = 5.35
nonempty_cycles/pkt = 93.8

At 6 cores, total throughput is unchanged but each worker receives fewer packets. Smaller batches and more empty polling push Other higher.

Reason: once total throughput is saturated around 115.9 Mpps, adding workers reduces per-core packet rate. Polling, timer, stats, branch, TX-buffer control, and queue/cache overhead are amortized over fewer packets.

In multi-core under this optimized scenario, the performance bottleneck has shifted away from software logic and to architecture friction (other) and hardware limitations

Table 1: Cycle Breakdown for Preserve Mode Across Different Core Counts

Cores	Throughput (Mpps)	Total (Cycles)	RX (Cycles)	Parse (Cycles)	Rewrite (Cycles)	Checksum (Cycles)	TX (Cycles)	Other (Cycles)
1	74.87	27.80	5.00	3.40	4.30	1.50	10.90	2.70
2	115.92	36.00	11.45	3.25	4.35	1.50	11.75	3.60
4	115.92	71.12	25.30	2.75	6.30	1.50	11.03	24.27
6	115.92	105.16	29.20	3.35	5.60	2.78	12.42	51.80
8	115.93	139.26	31.88	4.14	7.70	3.50	13.74	78.44

Optimization Results: By employing the Preserve strategy, checksum overhead was successfully reduced to 1.5 to 3.5 cycles. Furthermore, explicit processing stages, such as Parse, Rewrite, and TX, exhibited good stability under multi-core scaling, thereby validating the efficacy of the optimization methods

System Limits (Compared against the 100G's theory maximum value of 148.8 Mpps)

- The system reached a throughput ceiling of about 115.9 Mpps using just two cores, which is very close to 148.8 Mpps (78%)
- The per-packet RX cycle count increased from 5.0 cycles with one core to 31.88 cycles with eight cores.
- The Other cycle count increased significantly from 2.7 to 78.44

Current Bottleneck After All Optimizations

The bottleneck has moved from checksum to I/O path and control overhead

- Checksum preserve removes most checksum cost.
- Fast client shows the server-side benefit by eliminating input starvation.
- Ring/burst changes stabilize high-load tests.
- TX-buffer reduces high-core TX cost.
- RX prefetch reduces RX cost.
- But after line rate is reached, extra cores mainly increase small-batch polling/control overhead.

Current bottleneck: line-rate saturation + small bursts + empty polling + queue/NIC/cache shared-resource pressure.

Design Lessons for User-space Network Stacks

Lessons grounded in our measured optimizations

1. Instrument the datapath first.

Throughput alone is not enough; use stage cycles, drops, avg_burst, and empty_poll_ratio to define bottleneck.

2. Avoid unnecessary per-packet work.

Checksum preserve was highly effective for echo. Checksum offload could be more helpful for other benchmarks.

3. The traffic generator is part of the system.

A slow client hid the server-side performance improvement by creating empty polling.

4. Batching is essential but not free.

Larger rings/bursts and TX-buffering help stability and TX cost, but small batches still amplify control overhead.

5. Prefetch helps memory access, not scheduling or polling.

RX prefetch lowered RX cycles, but it did not solve high-core "Other".

6. More cores are not always better.

After reaching line rate, additional workers can reduce per-core batch size and increase cycles/pkt.



THANKS !